

**PROJECT REPORT**

# PYTHON MCQ QUIZ GAME

## *Control Flow and Functions*

|                      |                                  |
|----------------------|----------------------------------|
| Submitted By         | Gargi Sharma                     |
| Registration Number  | 26MIB10037                       |
| Project Type         | Console-Based Python Application |
| Programming Language | Python                           |

### **Academic Project Submission**

Academic Year: 2026-27

## CERTIFICATE

This is to certify that **Gargi Sharma**, Registration Number **26MIB10037**, has completed the Python Programming project titled **“Python MCQ Quiz Game - Control Flow and Functions”** as a part of academic coursework. The project demonstrates the use of Python data structures, conditional statements, loops, functions, user input, and basic result processing.

| Faculty / Project Guide Signature | Date  |
|-----------------------------------|-------|
| _____                             | _____ |

## DECLARATION

I, Gargi Sharma (Registration Number 26MIB10037), declare that this project report titled **“Python MCQ Quiz Game - Control Flow and Functions”** is prepared for academic submission. The program and explanations presented in this report are based on the Python implementation supplied for the project. The project is intended for learning and demonstration of fundamental Python programming concepts.

Gargi Sharma  
26MIB10037

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty and institution for providing the opportunity to develop this Python programming project. I also appreciate the learning resources and programming environment that helped in understanding control flow, loops, functions, lists, dictionaries, strings, user input, and basic arithmetic operations. This project provided practical experience in converting simple programming concepts into a working console application.

## TABLE OF CONTENTS

|                                    |    |
|------------------------------------|----|
| 1. Abstract                        | 6  |
| 2. Introduction                    | 6  |
| 3. Problem Statement               | 6  |
| 4. Objectives                      | 7  |
| 5. Scope of the Project            | 7  |
| 6. Tools and Technologies          | 8  |
| 7. System Requirements             | 8  |
| 8. Conceptual Design               | 9  |
| 9. Algorithm / Working Procedure   | 9  |
| 10. Program Modules                | 10 |
| 11. Important Python Concepts Used | 11 |
| 12. Testing and Validation         | 12 |
| 13. Sample Output                  | 10 |
| 14. Advantages and Limitations     | 11 |
| 15. Future Enhancements            | 11 |
| 16. Conclusion                     | 12 |
| 17. References                     | 12 |
| Appendix A. Source Code            | 13 |

## 1. ABSTRACT

The “Python MCQ Quiz Game” is a console-based quiz application developed using Python. The program presents ten multiple-choice questions covering fundamental concepts of control flow and functions. Each question has four options, and the user enters an answer using A, B, C, or D. The program checks the response against a predefined correct answer, updates the score, and finally calculates the number of correct answers, incorrect answers, percentage, and a performance message. The project is designed to demonstrate how basic Python constructs can be combined into a small, interactive application.

## 2. INTRODUCTION

Python is a high-level, general-purpose programming language known for its readable syntax and large ecosystem. For beginners, Python provides a convenient environment to learn programming logic before moving to larger applications. This project applies introductory Python concepts in a practical quiz scenario. The program is intentionally simple and text-based so that the focus remains on logic, data handling, and program flow rather than graphical interface development.

## 3. PROBLEM STATEMENT

The objective is to design a simple interactive quiz program that can present predefined multiple-choice questions, accept answers from a user, evaluate those answers, and display a final score. The program should maintain the score throughout the quiz and provide a percentage-based performance message at the end.

## 4. OBJECTIVES

- Create an interactive MCQ quiz using Python.
- Demonstrate the use of lists and dictionaries to store structured question data.
- Use for loops, if-elif-else statements, input(), and string operations in one application.
- Apply a user-defined function to organize the complete quiz logic.
- Calculate score, incorrect answers, and percentage automatically.
- Display a clear final result and performance message.

## 5. SCOPE OF THE PROJECT

The current project is limited to a single-player console quiz with ten predefined questions. It can be executed in a standard Python environment without external libraries. The design can later be extended to support larger question banks, randomized questions, difficulty levels, timers, persistent scores, a graphical interface, or a web-based quiz system.

## 6. TOOLS AND TECHNOLOGIES

|                      |                                       |
|----------------------|---------------------------------------|
| Programming Language | Python 3.x                            |
| Application Type     | Console / Command-Line Application    |
| Programming Style    | Function-based procedural programming |
| Core Data Structures | List and Dictionary                   |

|                    |                                |
|--------------------|--------------------------------|
| External Libraries | None required                  |
| Input Method       | Keyboard input through input() |
| Output Method      | Console using print()          |

## 7. SYSTEM REQUIREMENTS

### 7.1 Hardware Requirements

- Processor: Any modern dual-core processor or better
- RAM: 2 GB or more recommended
- Storage: Less than 50 MB for the source code and report
- Keyboard and display for user interaction

### 7.2 Software Requirements

- Operating System: Windows, Linux, or macOS
- Python 3.x installed and configured
- Any Python-compatible IDE or terminal (for example, IDLE, VS Code, PyCharm, or command prompt/terminal)

## 8. CONCEPTUAL DESIGN

The application follows a simple sequential structure. A function named `quiz_game()` contains the complete program logic. The question bank is represented by a list of dictionaries. Each dictionary stores one question, a list of four options, and the correct answer. A loop processes every question one by one. The user response is converted to uppercase using `upper()`, compared with the stored answer, and used to update the score. After the loop finishes, arithmetic operations are used to compute the final percentage.

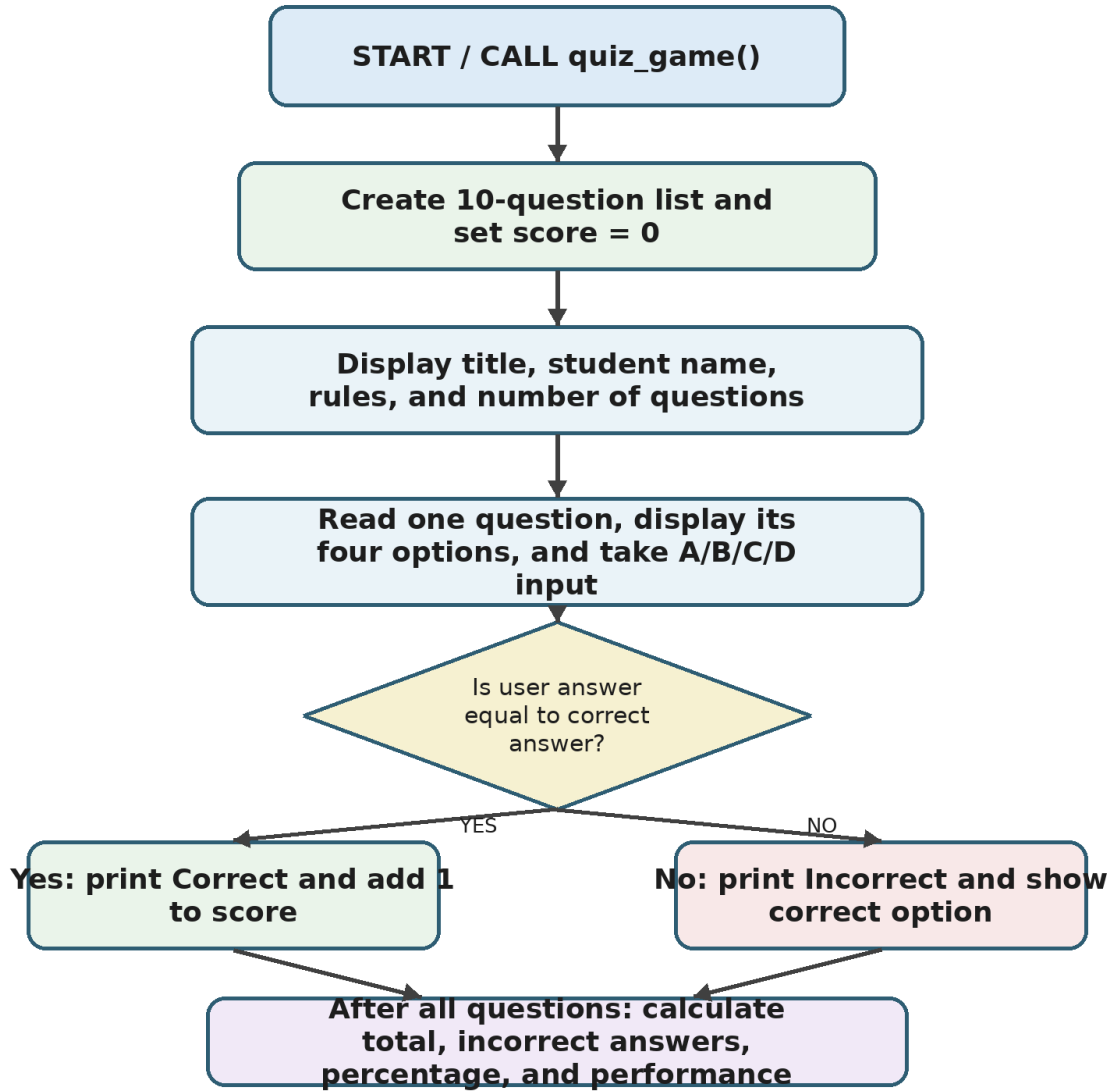


Figure 1. Flowchart of the Python MCQ Quiz Game

## 9. ALGORITHM / WORKING PROCEDURE

1. Define the quiz\_game() function.
2. Create a list named questions containing ten question dictionaries.
3. Initialize score to 0.
4. Display the project heading, student name, quiz rules, and number of questions.
5. Use enumerate(questions, 1) to process each question with a visible question number.
6. Display the question text and all four options.
7. Read the user response with input() and convert it to uppercase.
8. Compare the response with the stored correct answer.
9. For a correct response, display a success message and increment score by 1.
10. For an incorrect response, display an error message and show the correct option.
11. After all ten questions, calculate total questions, incorrect answers, and percentage.
12. Use if-elif-else conditions to display a performance message.
13. Print the final result and end the program.

## 10. PROGRAM MODULES

| Module                    | Purpose                                         | Key Code Elements           |
|---------------------------|-------------------------------------------------|-----------------------------|
| 1. Question Bank          | Stores questions, options, and correct answers. | questions, list, dictionary |
| 2. Quiz Initialization    | Starts the quiz and sets the initial score.     | score = 0                   |
| 3. Question Loop          | Displays each question and its options.         | for, enumerate(), print()   |
| 4. Input and Validation   | Reads and normalizes the answer.                | input(), upper()            |
| 5. Answer Evaluation      | Checks correctness and updates score.           | if/else, += 1               |
| 6. Result Calculation     | Computes totals and percentage.                 | len(), arithmetic operators |
| 7. Performance Evaluation | Maps percentage to a message.                   | if-elif-else                |
| 8. Program Entry          | Calls the main quiz function.                   | quiz_game()                 |

## 11. IMPORTANT PYTHON CONCEPTS USED

### 11.1 Function Definition and Function Call

```
def quiz_game():
    ...

quiz_game()
```

The function groups the complete quiz logic into one reusable unit. The final line invokes the function and starts execution.

### 11.2 List of Dictionaries

```
questions = [
    {
        "question": "...",
        "options": ["A) ...", "B) ...", "C) ...", "D) ..."],
        "answer": "B"
    }
]
```

A list stores all quiz questions, while each dictionary stores related fields for one question.

### 11.3 Looping with enumerate()

```
for number, q in enumerate(questions, 1):
    print("Question", number)
```

enumerate() generates both an index and the corresponding question item. Starting at 1 makes the displayed numbering user-friendly.

### 11.4 Conditional Statements

```
if user_answer == q["answer"]:
    score += 1
else:
    print("Incorrect answer!")
```

Conditional statements are used to evaluate the user response and later classify performance based on percentage.

### 11.5 Input Normalization

```
user_answer = input("Enter your answer (A/B/C/D): ").upper()
```

upper() ensures that lowercase entries such as b are converted to uppercase before comparison.

### 11.6 Percentage Calculation

```
percentage = (score / total_questions) * 100
```

The percentage is calculated by dividing the correct score by the total number of questions and multiplying by 100.

## 12. TESTING AND VALIDATION

The following test cases represent the expected behavior of the supplied program.

| Test Case | Input / Condition               | Expected Result                                                | Status |
|-----------|---------------------------------|----------------------------------------------------------------|--------|
| TC-01     | Correct option entered          | “Correct answer!” displayed and score increases by 1           | Pass   |
| TC-02     | Incorrect option entered        | “Incorrect answer!” displayed and correct option shown         | Pass   |
| TC-03     | Lowercase input such as b       | Input converted to uppercase and evaluated normally            | Pass   |
| TC-04     | All 10 answers correct          | Final score 10/10 and 100%                                     | Pass   |
| TC-05     | All 10 answers incorrect        | Final score 0/10 and 0%                                        | Pass   |
| TC-06     | Mixed correct/incorrect answers | Score, incorrect count, and percentage match the answers given | Pass   |

### 12.1 Time Complexity of the Quiz

Let  $n$  be the number of questions and  $k$  be the maximum number of options per question. Each question displays its options and performs a constant amount of comparison work, so the quiz runs in  $O(nk)$ . Because  $k = 4$  in the supplied implementation, this simplifies to  $O(n)$ . The memory required for the question bank is  $O(nk)$ , which is  $O(n)$  for a fixed four-option format.

## 13. SAMPLE OUTPUT

A representative run of the program is shown below. The exact final score depends on the answers entered by the user.

```
=====
PYTHON MCQ QUIZ GAME
Control Flow and Functions
```

## PYTHON MCQ QUIZ GAME - PROJECT REPORT

```
=====
Name: Gargi Sharma
=====

Welcome to the Python Quiz!
There are 10 questions.
Each correct answer gives 1 mark.
There is no negative marking.

-----
Question 1
-----
Which statement is used to make a decision in Python?
A) for
B) if
C) def
D) return

Enter your answer (A/B/C/D): B
Correct answer!

...

=====
                        QUIZ RESULT
=====
Name: Gargi Sharma
Total Questions: 10
Correct Answers: 8
Incorrect Answers: 2
Final Score: 8 / 10
Percentage: 80.0 %
Performance: Excellent!
=====
                        Thank you for playing!
=====
```

## 14. ADVANTAGES AND LIMITATIONS

### 14.1 Advantages

- Simple and easy-to-understand console interface.
- No external libraries are required.
- Demonstrates several fundamental Python concepts in one program.
- Provides immediate feedback after each answer.
- Automatically calculates the final score and percentage.

### 14.2 Limitations

- The questions and answers are hard-coded in the source file.
- Only one player can attempt the quiz at a time in the current version.
- The program accepts A/B/C/D format conceptually, but it does not explicitly reject or re-prompt for other inputs.
- There is no question randomization, timer, database, login system, or persistent score history.
- The application is console-based and does not provide a graphical user interface.

## 15. FUTURE ENHANCEMENTS

- Add input validation with a loop so only A, B, C, or D are accepted.
- Randomize the order of questions and options.

- Create question categories and difficulty levels.
- Store questions in a JSON, CSV, or database file instead of hard-coding them.
- Add a timer for each question and an overall quiz timer.
- Develop a GUI using Tkinter or a web interface using a Python web framework.
- Maintain player names, high scores, and attempt history.
- Add explanations for the correct answers to support learning.

## 16. CONCLUSION

The Python MCQ Quiz Game successfully combines basic programming concepts into a functional interactive application. The project demonstrates practical use of functions, lists, dictionaries, loops, conditional statements, string handling, input/output, and arithmetic calculations. Although the application is intentionally small, its structure provides a foundation for developing more advanced quiz systems with persistent data, randomization, timing, and graphical or web interfaces.

## 17. REFERENCES

- Python documentation: Python 3 Language Reference and tutorial, <https://docs.python.org/3/>
- Python built-in functions documentation, <https://docs.python.org/3/library/functions.html>
- Python tutorial topics: control flow tools, data structures, and defining functions, <https://docs.python.org/3/tutorial/>

## APPENDIX A. SOURCE CODE

```
def quiz_game():
    # Quiz questions
    questions = [
        {
            "question": "Which statement is used to make a decision in Python?",
            "options": [
                "A) for",
                "B) if",
                "C) def",
                "D) return"
            ],
            "answer": "B"
        },
        {
            "question": """"What will be the output?

x = 10

if x > 5:
    print("Yes")
else:
    print("No")""",
            "options": [
                "A) No",
                "B) Error",
                "C) Yes",
                "D) 10"
            ],
            "answer": "C"
        },
        {
            "question": "Which loop is commonly used when we know how many times we want to
repeat something?",
            "options": [
                "A) if",
                "B) while",
                "C) for",
                "D) def"
            ],
            "answer": "C"
        },
        {
            "question": "What does the break statement do?",
            "options": [
                "A) Skips the current iteration",
                "B) Stops the loop",
                "C) Starts the loop again",
                "D) Defines a function"
            ],
            "answer": "B"
        },
        {
            "question": "What does the continue statement do?",
            "options": [
                "A) Stops the program",
                "B) Stops the loop completely",
                "C) Skips the current iteration and continues with the next one",
                "D) Defines a new function"
            ],
            "answer": "C"
        }
    ],
```

## PYTHON MCQ QUIZ GAME - PROJECT REPORT

```
{
    "question": "Which keyword is used to define a function in Python?",
    "options": [
        "A) function",
        "B) func",
        "C) define",
        "D) def"
    ],
    "answer": "D"
},

{
    "question": """"What will be the output?

def add(a, b):
    return a + b

print(add(3, 4))""",
    "options": [
        "A) 3",
        "B) 4",
        "C) 7",
        "D) 12"
    ],
    "answer": "C"
},

{
    "question": """"In the following function, what are a and b called?

def multiply(a, b):
    return a * b""",
    "options": [
        "A) Arguments",
        "B) Parameters",
        "C) Operators",
        "D) Variables only"
    ],
    "answer": "B"
},

{
    "question": "What is the purpose of the return statement in a function?",
    "options": [
        "A) To stop every loop in the program",
        "B) To send a value back from the function",
        "C) To define a function",
        "D) To print a value automatically"
    ],
    "answer": "B"
},

{
    "question": """"What will be the output?

for i in range(3):
    print(i)""",
    "options": [
        "A) 1 2 3",
        "B) 0 1 2",
        "C) 0 1 2 3",
        "D) 3 2 1"
    ],
    "answer": "B"
}
}

# Initial score
score = 0
```

## PYTHON MCQ QUIZ GAME - PROJECT REPORT

```
# Project heading
print("=" * 60)
print("                PYTHON MCQ QUIZ GAME")
print("                Control Flow and Functions")
print("=" * 60)
print("Name: Gargi Sharma")
print("=" * 60)

print("\nWelcome to the Python Quiz!")
print("There are 10 questions.")
print("Each correct answer gives 1 mark.")
print("There is no negative marking.")

# Loop through all questions
for number, q in enumerate(questions, 1):

    print("\n" + "-" * 60)
    print("Question", number)
    print("-" * 60)

    print(q["question"])

    # Display options
    for option in q["options"]:
        print(option)

    # Take answer from user
    user_answer = input("\nEnter your answer (A/B/C/D): ").upper()

    # Check the answer
    if user_answer == q["answer"]:
        print("Correct answer!")
        score += 1
    else:
        print("Incorrect answer!")
        print("Correct answer is:", q["answer"])

# Calculate result
total_questions = len(questions)
incorrect_answers = total_questions - score
percentage = (score / total_questions) * 100

# Display final result
print("\n" + "=" * 60)
print("                QUIZ RESULT")
print("=" * 60)

print("Name:", "Gargi Sharma")
print("Total Questions:", total_questions)
print("Correct Answers:", score)
print("Incorrect Answers:", incorrect_answers)
print("Final Score:", score, "/", total_questions)
print("Percentage:", percentage, "%")

# Performance message
if percentage >= 80:
    print("Performance: Excellent!")
elif percentage >= 60:
    print("Performance: Good!")
elif percentage >= 40:
    print("Performance: Keep Practicing!")
else:
    print("Performance: Need More Practice!")

print("=" * 60)
print("                Thank you for playing!")
print("=" * 60)
```

## PYTHON MCQ QUIZ GAME - PROJECT REPORT

```
# Start the quiz  
quiz_game()
```